NORTHERN ARIZONA UNIVERSITY

School of Informatics, Computing, and Cyber Systems

CS501
PROGRAMMING: PYTHON BASICS

Introduction to Python

Dr. Igor Steinmacher

e-mail: Igor.Steinmacher@nau.edu

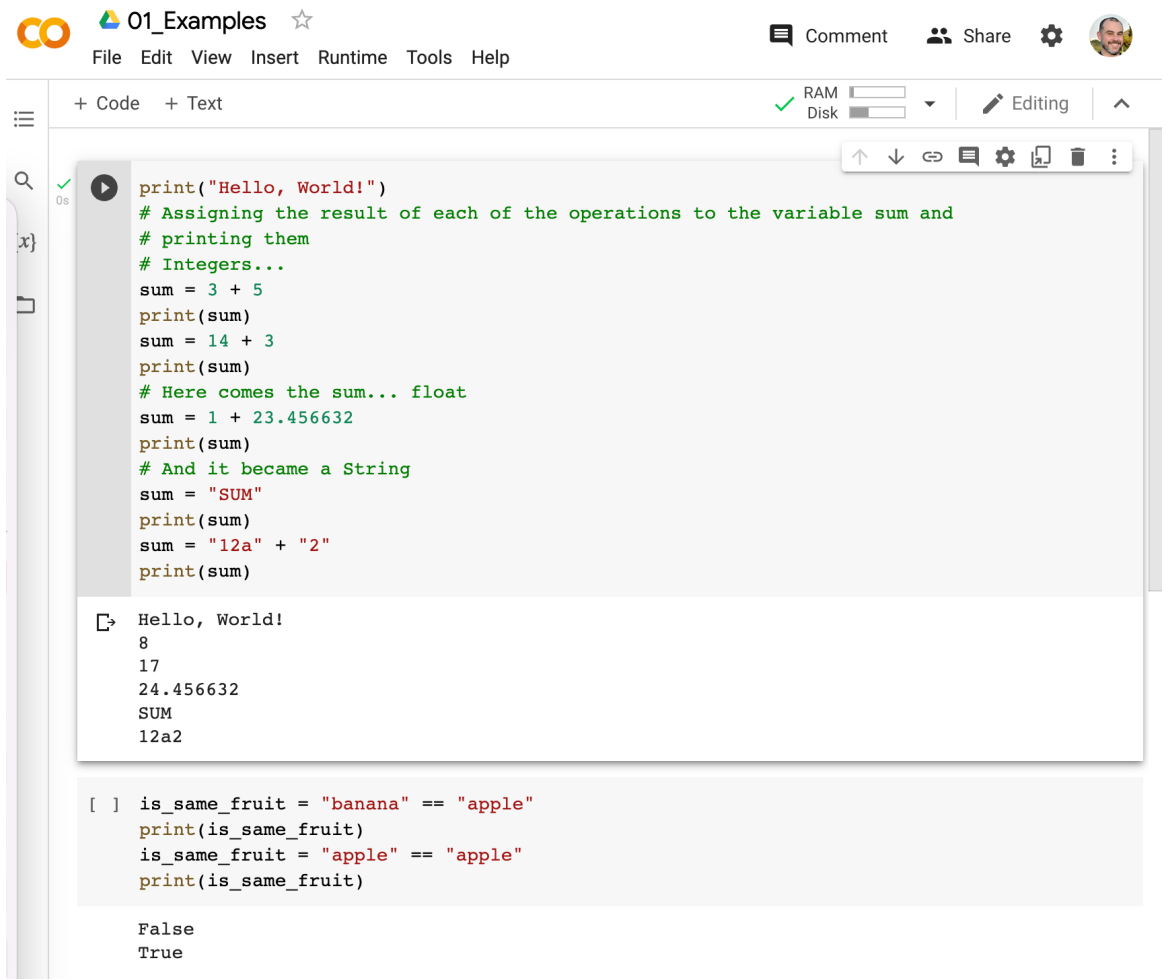
Twitter: @igorsteinmacher

Remembering: Formatting

- Python uses **indentation** to delimit blocks of code
- Comments start with #
- Colons ':' are used to start a new block for different constructs

```
#function that answers if a number is even or not
def isEven (number):
    #is the remainder when dividing number by 2 equals to 0
    if (number % 2 == 0):
        #yes, the number is even
        return True
    return False
```

FROM NOW ON: GOOGLE COLLABORATORY IS PREFERRED



https://colab.research.google.com/

All online

Shareable

colab

Let's talk about code styles: PEP-8

- PEP 8 is for Python code on Python projects
  - **https://peps.python.org/pep-0008/**
  - **https://pep8.org/**

- If the style causes more trouble than benefit, this is a bad smell

- Consistency is key: keep the style from older code, or change it all

PEP8: INDENTATION

- Indentation should always be four (4) spaces long.
  - **Python 3.x: you can use spaces or tabs**
  - **Make sure that you are consistent with your tabs and spaces**

- For long lines (>79 characters):
  - **Align the first letter of subsequent lines with the first character of the original line.**
  - **Function definitions should use double indents for subsequent lines.**

```
# No extra indentation.
if (this_is_one_thing and
    that_is_another_thing):
    do_something()

# Add a comment, which will provide some distinction in editors
# supporting syntax highlighting.
if (this_is_one_thing and
    that_is_another_thing):
    # Since both conditions are true, we can frobnicate.
    do_something()

# Add some extra indentation on the conditional continuation line.
if (this_is_one_thing
        and that_is_another_thing):
    do_something()
```

PEP8: PARENTHESIS AND BRACKETS

- End parenthesis and brackets should be on their own lines.
- End parenthesis and brackets should either be aligned to
  - **the first non-whitespace character of the last line**
  - **or the first non-whitespace character of the construct itself.**

```
my_list = [
    1, 2, 3,
    4, 5, 6,
    ]
result = some_function(
    'a', 'b', 'c',
    'd', 'e', 'f',
    )
```

```
my_list = [
    1, 2, 3,
    4, 5, 6,
]
result = some_function(
    'a', 'b', 'c',
    'd', 'e', 'f',
)
```

PEP8: NAMING CONVENTIONS

- Never use the characters 'l' (lowercase letter el), 'O' (uppercase letter oh), or 'I' (uppercase letter eye) as single character variable names.
- Class names should normally use the CapWords convention.
- Function names and instance variables should be lowercase with underscores as necessary (var_name)
- Constants should be ALL_CAPITAL_WITH_UNDERSCORES.

PEP8: Encoding

- Code in the core Python 3.x distribution should always use **UTF-8**

Types types types: Strings

- Strings are delimited by single or double quotation marks

```
>>> single_quote = 'python'
>>> double_quote = "python"
>>> I_want_the_quote = 'It\'s python'
>>> yes_the_quote = "It's python"
```

Types types types

- Python has five standard data types
  - **Numbers**

```
int     #signed integers: -20, -1, 0, 12342
long    #long integers: 0122L, -0x19ACL
float   #real values: 1.23, -78.12, 32.3+e18
complex #complex numbers: 45.j, 3.14j
```

  - **String**

```
str     #sequence of chars: 'abc', '6 days', '_-_'
```

  - **List**

```
list     # ['abcd', 786 , 2.23, 'john', 70.2]
```

  - **Tuple**

```
tuple    # ('abcd', 786 , 2.23, 'john', 70.2)
```

  - **Dictionary**

```
dict     # {'name': 'john','code':6734, 'dept': 'sales'}
```

  - **Boolean**

STRINGS

- Strings in Python are a contiguous set of characters represented in the quotation marks.

- Python allows for either pairs of single (' ') or double (' ') quotes.

```
>>> str = 'this is a string'
>>> another_str = 'this is also a string'

>>> print (str)
this is a string
```

STRINGS

- Operations using Strings

```
#operations using strings
>>> salutation = 'Hello'
>>> name = 'John'
>>> complete_salut = salutation + ', ' + name + '!'
>>> print (complete_salut)
Hello, John!
>>> real_long_string = 'this is a long string. '\
                       'It has multiple parts, '\
                       'but all in one line. '

>>> print (salutation[0]) # Prints first character of the string
H
>>> print (salutation[2:5]) # Prints characters starting from 3rd to 5th
llo
>>> print (salutation*2) # Prints string two times
HelloHello
```

WE NEED TO INPUT TEXT, RIGHT?

- Getting user inputs is usually important

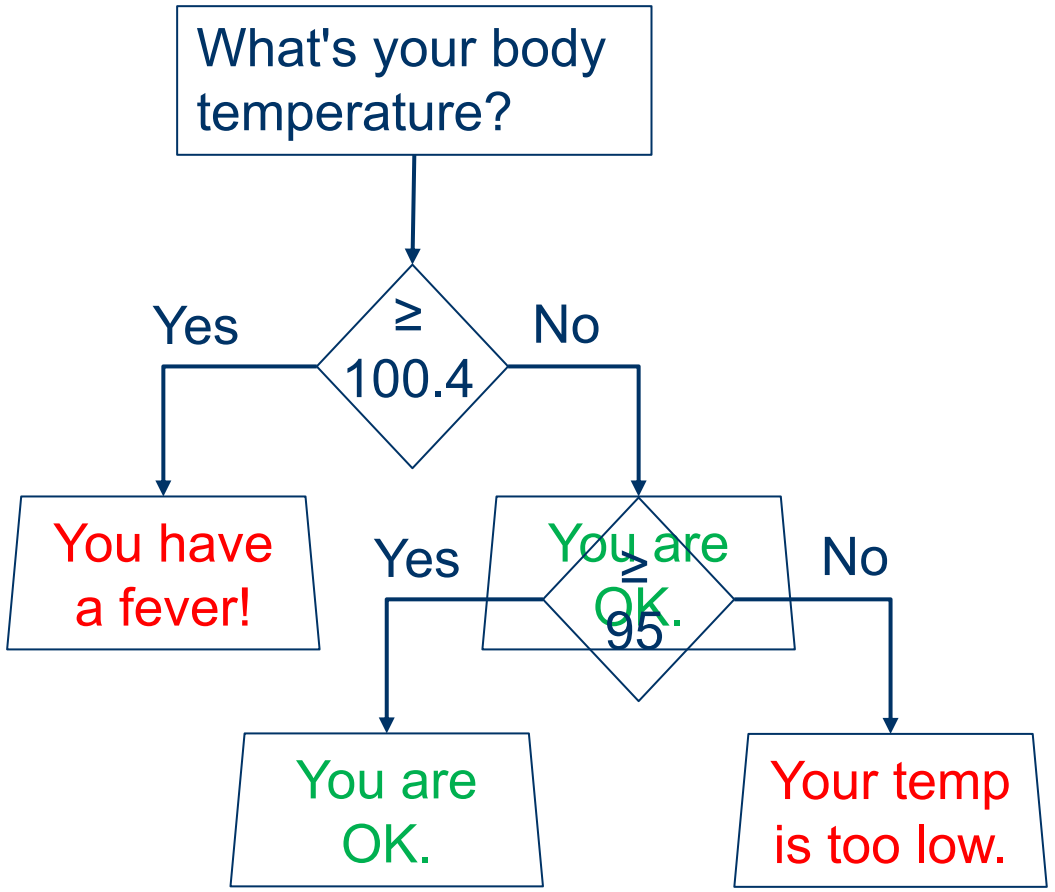
```
#input() function waits for an input from the keyboard
>>> salutation = 'Hello'
>>> name = input('Tell me your name: ')
Tell me your name: <INPUT + ENTER>
>>> complete_salut = salutation + ', ' + name + '!'
>>> print (complete_salut)
Hello, <NAME ENTERED>!

>>> weight = input('Enter your weight in lb: ')
Enter your weight in lb: 200
>>> weight = int(weight) # what am I doing here???
>>> weight_kg = weight/2.205
>>> print (weight_kg)
90.70294784580499
```

5-MINUTE MADNESS

- **Write a program to prompt the user for hours and rate per hour to compute gross pay.**
  - **Enter Hours: 35**
  - **Enter Rate: 2.75**
  - **Pay: 96.25**

- **Write a program which prompts the user for a Celsius temperature, convert the temperature to Fahrenheit, and print out the converted temperature.**

CONDITIONAL




```
temperature = float(input('Type your temp'))
if (temperature >= 100.4):
    print ('You have a fever')
elif (temperature >= 95):
    print ('You are OK.')
else:
    print ('Your temp is too low')
```

CONDITIONAL

- if – else (given two numbers, tell me what is the greater)

```
if (x > y):
   print ('x is greater than y')
elif (y < x):
   print ('y is greater than x')
else:
   print ('they are equal!')
```

```
parity = 'even' if x % 2 == 0 else 'odd'
```

Comparison Operations

Operation	Meaning
<	strictly less than
<=	less than or equal
>	strictly greater than
>=	greater than or equal
==	equal
!=	not equal
is	object identity
is not	negated object identity

Recap: Logical Operators

- AND

```
>> True and True
True
>> (2 > 1) and (4 == 4)
True
>> (7 < 4) and (10 != 9)
False
```

- OR

```
>> False or True
True
>> (1 >= 1) or (2 == 4)
True
>> (7<4) or (10 != 9)
True
```

- NOT

```
>> not True
False
>> not (2 > 1)
False
>> not ((7 < 4) and (10 != 9))
True
```

3-MINUTE MADNESS

- **Write a program to prompt the user age and returns:**
  - **'You are a kid' → in case the person is 12 or younger**
  - **'Teenager around' → If the age is from 13 to 19**
  - **'An adult, eh?' → If older than 19**

- Do it in a file called age.py and run it (or in a collab notebook)

SOME MORE PEP8 TALK

- Avoid extraneous whitespace in the following situations:
  - **Immediately inside parentheses, brackets or braces**
  - **Between a trailing comma and a following close parenthesis:**
  - **Immediately before a comma, semicolon, or colon**

```
#Yes:
spam(ham[1])

#No:
spam( ham[ 1 ] )

#Yes:
foo = (0,)

#No:
bar = (0, )

#Yes:
if x == 4: print x, y; x, y = y, x

#No:
if x == 4 : print x , y ; x , y = y , x
```

And more

- Create a program that, given the age, weight and height of someone, returns the BMI (Body Mass Index) interpretation according to CDC (Center for Disease and Control)

- For adults (20 Years old or older), the formula is:
  BMI = weight (kg) / [height (m)]^2
- Your program must present the following interpretation, given the calculated BMI
  - Below 18.5 = **Underweight**
    18.5-24.9 = **Normal or Healthy Weight**
    25.0-29.9 = **Overweight**
    30.0 or Above = **Obese**

- For children and teens (younger than 20), the program must provide the following output: 'BMI is age- and sex-specific and we cannot calculate'

NORTHERN ARIZONA UNIVERSITY

School of Informatics, Computing, and Cyber Systems

LISTS: WHAT PYTHON IS KNOWN FOR

Lists

- A Python list is an **ordered**, **mutable** sequence of objects
- Lists can contain a mixture of any sort of object: numbers, Booleans, strings, other lists, …
- Lists can grow or shrink

```
>>> list_of_numbers = [1,2,3,4,5]
>>> len(list_of_numbers)
5
>>> list_of_numbers[0]
1
>>> list_of_numbers[4]
5
>>> list_of_numbers[-2]
4
```

Lists

```
#extending it
>>> list_of_numbers.extend([6,7,8])
>>> print(list_of_numbers)
[1,2,3,4,5,6,7,8]

#slicing it
>>> piece = list_of_numbers[:4] #beginning to 4
>>> print (piece)
[1,2,3,4]
>>> piece = list_of_numbers[2:6] #from position 2 to 6
>>> print (piece)
[3,4,5,6]

#shrinking it
>>> del list_of_numbers [2:5]
>>> print(list_of_numbers)
[1,2,6,7,8]
```

Lists

```
#merging
>>> list1 = [1,2,3]
>>> list2 = [4,5,6]
>>> list3 = list1 + list2
>>> print(list3)
[1,2,3,4,5,6]
>>> list1.extend(list2)
>>> print(list1)
[1,2,3,4,5,6]

#sorting
>>> list1 = [-1,4,0,9,2,7]
>>> list1.sort()
>>> print (list1)
[-1,0,2,4,7,9]
```

```
#other functions
>>> list1.append(<value>)
>>> list1.insert(<pos>,<value>)
>>> list1.remove(<value>)
>>> list1.clean()
>>> list1.remove(<value>)
>>> list1.index(<value>)
>>> list1.reverse()
>>> list1.set()
>>> <value> in list1
```

ITERATIONS (LOOPS)

- Starting with the **for** (highly used with iterable objects)

```
list_of_numbers = [1,2,3,4,5]

for element in list_of_numbers:
    print (element)
```

- But… also for counting

```
for x in range(10): #will iterate from 0 to 9
    if (x%3==0):
        continue
    print (x)

#what is the output?
```

ITERATIONS (LOOPS)

- Now iterating over a list of lists

```
# grades is a list that stores the name and the grade
# of students. Each position of the list has one list
# with two positions. The first stores the name, and
# the second stores the grade

grades = [['John',60],['Paul', 84],['Ben', 70], ['Tony',35]]
for entry in grades:
   if (entry[1]>60):
      print(entry[0] + ': approved')
   else:
      print(entry[0] + ': talk to the instructor')
```

ITERATIONS (LOOPS)

- Now iterating over a list of lists

```
# grades is a list that stores the name and the grade
# of students. Each position of the list has one list
# with two positions. The first stores the name, and
# the second stores the grade

grades = [['John',60],['Paul', 84],['Ben', 70], ['Tony',35]]
for name, grade in grades:
   if (grade>60):
      print(name + ': approved')
   else:
      print(name + ': talk to the instructor')
```

ITERATIONS (LOOPS)

- Now iterating over a list of lists

```
# grades is a list that stores the name and the grade
# of students. Each position of the list has one list
# with two positions. The first stores the name, and
# the second stores the grade

grades = [['John',60],['Paul', 84],['Ben', 70], ['Tony',35]]
for name, grade in grades:
    if (grade>60):
        print(name + ': approved')
    else:
        print(name + ': talk to the instructor')
```

Iterations (Loops)

- Using While Loop

```
i = 1
while i <= 10:
    print(i)
    i += 1
```

Hands on time

- I will create a program that stores a set of numbers in a list.
- The list will be filled with random numbers. The user needs to be able to:
  1. **Add a number to the list**
  2. **Present the number of elements in the list, and list the numbers (one per line)**
  3. **Clear the list**
  4. **Exit the application**

PRACTICE TIME

NORTHERN ARIZONA UNIVERSITY

School of Informatics, Computing, and Cyber Systems

OTHER TYPES OF COLLECTIONS

TUPLES

- A Python tuple is an ordered, **immutable** sequence of objects
- Like lists, but cannot be altered
  - **Do not have methods like reverse(), sort()**

```
>>> my_tuple = (6,34,6,7,2)
>>> len(mytuple)
5
>>> my_tuple[3]
7
>>> my_tuple[1:4]
[34, 6, 7]
>>> my_tuple[2]=8
Traceback (most recent call last):
  File '<stdin>', line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

10-MINUTE MADNESS

- Write a program which repeatedly reads integers until the user enters 'done'. Once 'done' is entered, print out the total, count, and average of the numbers. If the user enters anything other than an integer, print an error message and skip to the next number.
  - **Hint: isinstance (variable, int) → returns True if the variable is an integer**

- Rewrite the program above to use the following list instead of prompting the user:
  - **list_numbers=[2,6,2,5,8,2,7,3,5,3,5,7]**

DICTIONARIES

- A dictionary associates values with unique keys
  - **Use braces instead of square brackets**

```
>>> grades = {'John': 60, 'Paul': 84, 'Ben': 70, 'Tony': 35}
>>> print(grades['John'])
60
>>> grades['Kate'] = 100              # adds another entry
>>> grades['John'] = 90               # changes John's grade
>>> print(grades)
{'John': 90, 'Paul': 84, 'Ben': 70, 'Tony': 35, 'Kate': 100}
```

DICTIONARIES

```
>>> 'John' in grades
True
>>> 'Igor' in grades
False
>>> grades.get('Joel', -1) # will return -1 (avoid errors)
>>> grades.get('John', -1) # will return 90 (exists)

>>> all_keys = grades.keys().    # return a list of all keys
>>> all_values = grades.values() # return a list of all values
>>> all_pairs = grades.items()   # a list of (key, value) tuples
```

DICTIONARIES

```
# Nested structures... why not?
family = {'John': {'age': 30, 'weight': 170, 'city': 'Flagstaff'},
          'Paul': {'age': 45, 'weight': 200, 'city': 'Buenos Aires'},
          'Anna': {'age': 26, 'weight': 130, 'city': 'Paris'}}
# Lazy iteration
for member in family:
   print(member)

for member in family:
   print(family[member])
```

10-MINUTE MADNESS

- Given the grades dictionary provided in the dictionaries slide

```
{'John': 90, 'Paul': 84, 'Ben': 70, 'Tony': 35, 'Kate': 100}
Create a program that asks the user to provide a name:
1. If the name exists, you must delete the entry;
2. If it does not exist, you must ask for a grade and add the new
   student and the grade to the dict.
3. List all the grades in the dictionary after the deletion or
   inclusion
```

  - Yes, the program need to run just once, the point is testing dict manipulation